

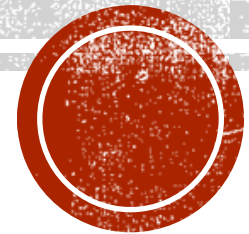
TP N°5

ASP.NET CORE WEB API

GESTION DES EMPLOYÉS

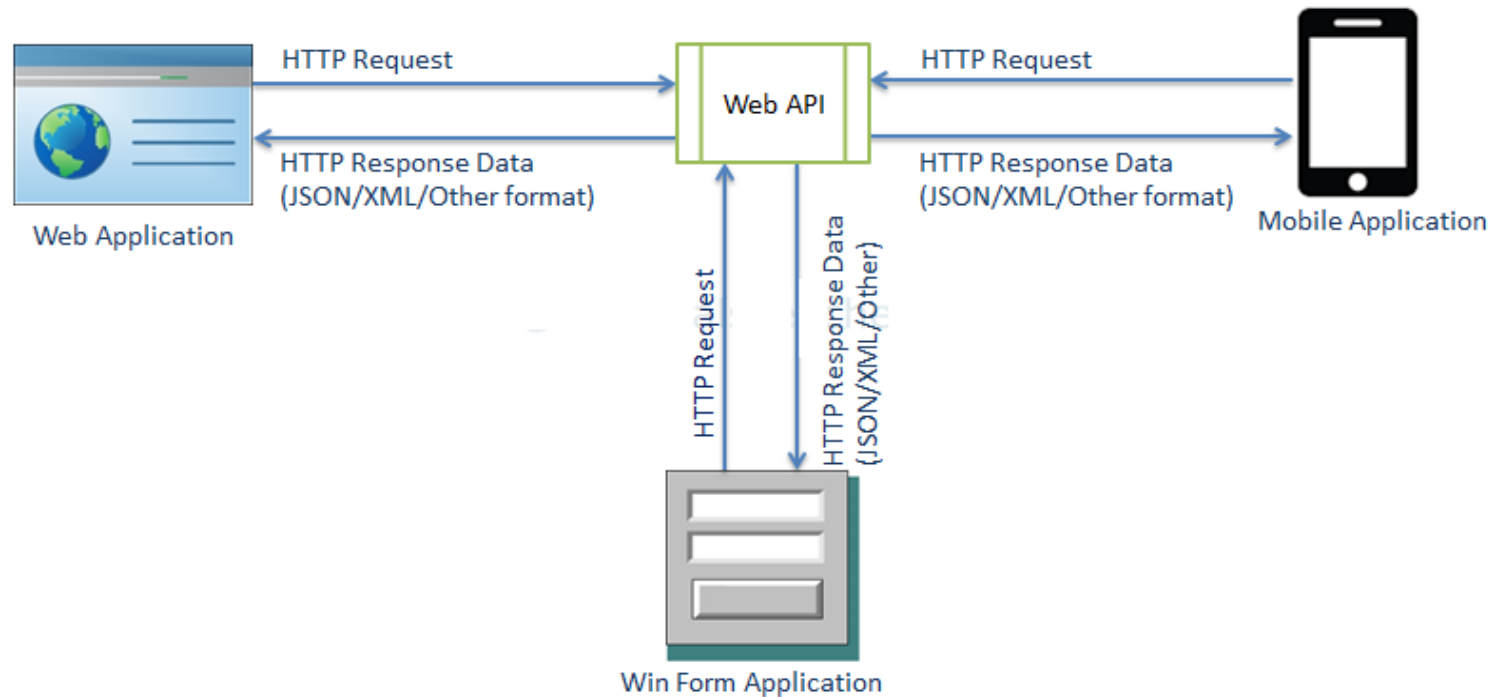
Enseignants :

Malek Zribi & Lilia Ayadi



INTRODUCTION

- Le Framework web ASP.NET Core MVC permet de créer des services web HTTP accessibles depuis n'importe quel client : navigateurs, appareils mobiles, applications IOT et autres.
- C'est une plate-forme idéale pour créer des services RESTful sous .NET Core.



RESTFULL API

What are RESTful APIs

RE **RE**presentational

S **S**tate

T **T**ransfer

A **A**pplication

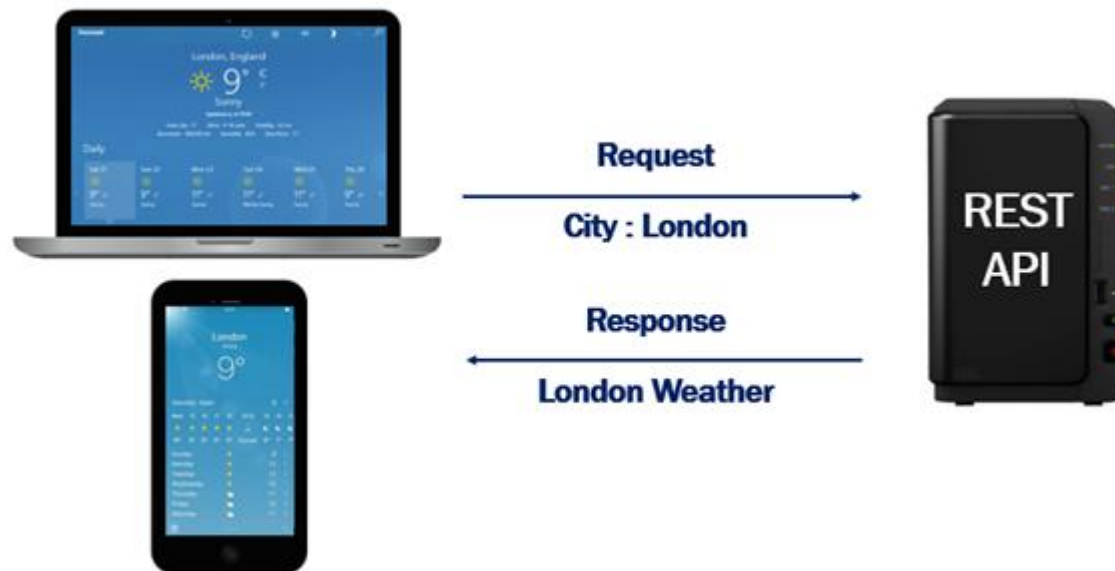
P **P**rogramming

I **I**nterface



EXEMPLE WEB API

- une API REST permet aux applications d'interagir les unes avec les autres et d'échanger des données.
- Par exemple, disons que vous créez une application mobile ou une application Web. Dans cette application, vous souhaitez afficher des données météorologiques telles que la température, l'humidité, la vitesse du vent, etc.



CONVENTIONS POUR LES WEB API

- Dans le contexte d'une API REST, une ressource est une entité de données telle que Produit, Employé, Client, Commande, etc.
- Par exemple, une API REST qui fournit des données sur les employés rend la liste des employés disponible à l'URI (Uniform Resource Identifier) suivant:

Protocol ↑
http:// **localhost:65180** **/api/employees**
↓
Domain Name

Optional ↑
/api/employees
↓
Endpoint



CONVENTIONS POUR LES WEB API

- En plus de l'URI, nous devons également envoyer un HTTP Verb au serveur. Voici les HTTP Verb courants:

- **GET**
- **POST**
- **PUT**
- **PATCH**
- **DELETE**

HTTP Verbs

CRUD Operations

- | | | |
|-----------------------|---|-----------------|
| ➤ Post | → | ➤ Create |
| ➤ Get | → | ➤ Read |
| ➤ Put or Patch | → | ➤ Update |
| ➤ Delete | → | ➤ Delete |

- C'est ce verbe HTTP qui indique à l'API quoi faire avec la ressource, par exemple, créer un nouvel employé, consulter, supprimer ou modifier un employé existant.



CONVENTIONS POUR LES WEB API

- la combinaison de l'URI et du verbe HTTP, qui est envoyé avec chaque requête, indique au serveur (c'est-à-dire à l'API REST) ce qu'il doit faire avec la ressource. Par exemple,

URI	HTTP Verb	Outcome
.../api/employees	GET	Gets list of employees
.../api/employees/1	GET	Gets employee with Id = 1
.../api/employees	POST	Creates a new employee
.../api/employees/1	PUT	Updates employee with Id = 1
.../api/employees/1	DELETE	Deletes employee with Id = 1



CRÉER UN PROJET WEB API

Créer un projet

Modèles de projet récents

-  API web ASP.NET Core C#
-  Application WebAssembly Blazor C#
-  Bibliothèque de classe C#
-  Nouvelle solution
-  Application web ASP.NET Core (modèle-vue-contrôleur) C#
-  ASP.NET Core vide C#
-  Application Windows Forms (.NET Framework) C#
-  Application Windows Forms C#

Rechercher des modèles (Alt+S)

Tout effacer

C# Toutes les plateformes Web

**Application web ASP.NET Core (modèle-vue-contrôleur)**
Modèle de projet permettant de créer une application ASP.NET Core avec des exemples de vues et de contrôleurs ASP.NET Core MVC. Vous pouvez également utiliser ce modèle pour les services HTTP RESTful.
C# Linux macOS Windows Cloud Service Web

**Application Blazor Server**
Modèle de projet permettant de créer une application Blazor Server qui s'exécute côté serveur dans une application ASP.NET Core, et qui gère les interactions utilisateur via une connexion SignalR. Vous pouvez utiliser ce modèle pour les applications web ayant des IU (interfaces utilisateur) dynamiques riches.
C# Linux macOS Windows Cloud Web

**API web ASP.NET Core**
Modèle de projet permettant de créer une application ASP.NET Core avec un exemple de contrôleur pour un service HTTP RESTful. Vous pouvez également utiliser ce modèle pour les vues et contrôleurs ASP.NET Core MVC.
C# Linux macOS Windows Cloud Service Web

**Service gRPC ASP.NET Core**
Modèle de projet pour la création d'un service gRPC ASP.NET Core.
C# Linux macOS Windows Cloud Service Web

Retour

Suivant



CRÉER UN PROJET WEB API

Configurer votre nouveau projet

Application web ASP.NET Core

C#

Linux

macOS

Windows

Cloud

Service

Web

Nom du projet

EmployeeWebAPI

Emplacement

C:\Users\USER\source\repos

Nom de la solution 

EmployeeWebAPI

☐

Placer la solution et le projet dans le même répertoire



CRÉER UN PROJET WEB API

Vérifier que votre application est d'infrastructure : .NET 8.0, afin de pouvoir tester directement avec Swagger intégré dans le projet

Informations supplémentaires

API web ASP.NET Core C# Linux macOS Windows API Cloud Service Web Web API

Infrastructure ⓘ

.NET 8.0 (Prise en charge à long terme)

Type d'authentification ⓘ

Aucun

☒ Configurer pour HTTPS ⓘ

☐ Activer la prise en charge du conteneur ⓘ

Conteneur OS ⓘ

Linux

Type de build du conteneur ⓘ

Dockerfile

☒ Activer la prise en charge d'OpenAPI ⓘ

☐ N'utilisez pas d'instructions de niveau supérieur. ⓘ

☒ Utiliser des contrôleurs ⓘ

☐ Inscrire dans l'orchestration de .NET Aspire ⓘ

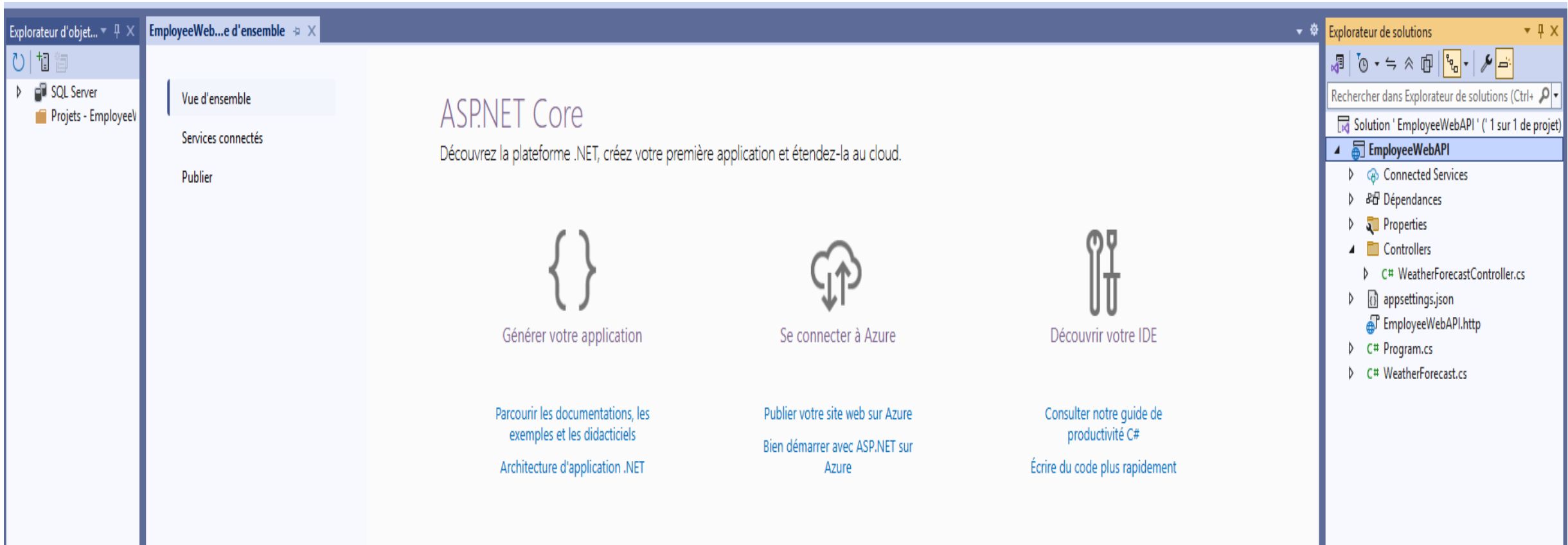
Version Aspire ⓘ

9.0



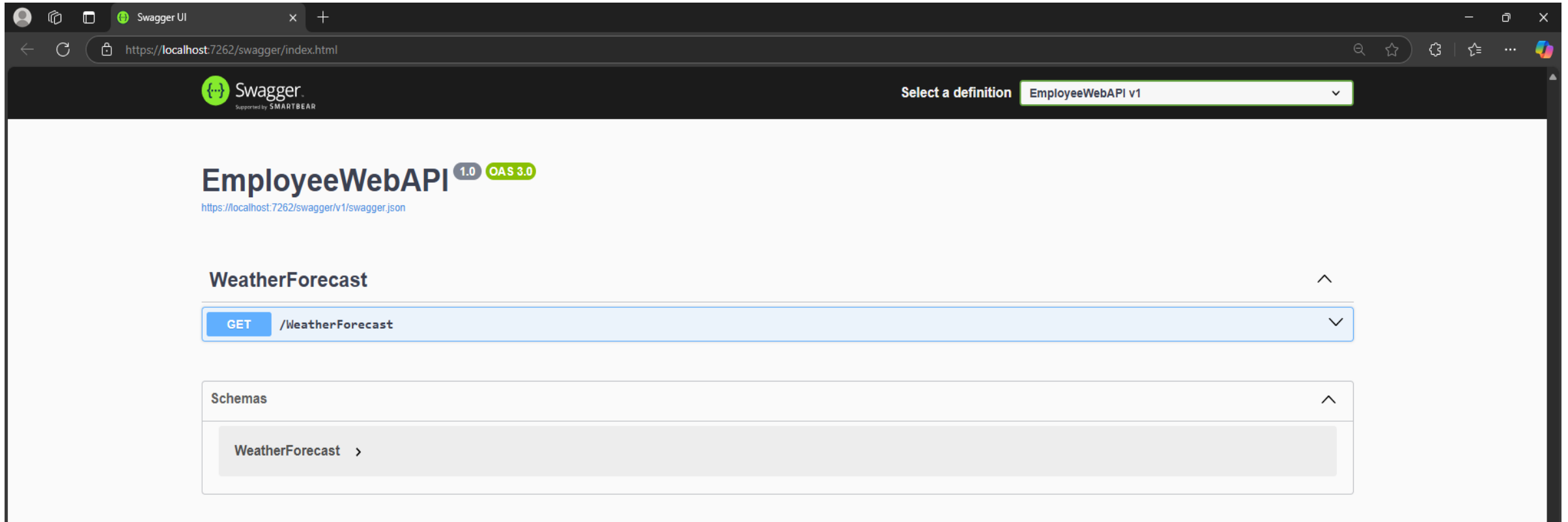
CRÉER UN PROJET WEB API

Votre projet aura la forme suivante :



CRÉER UN PROJET WEB API

Exécuter votre application pour avoir :



The screenshot displays the Swagger UI interface in a web browser. The browser's address bar shows the URL `https://localhost:7262/swagger/index.html`. The Swagger logo is visible in the top left, and a dropdown menu in the top right allows selecting the API definition, currently set to "EmployeeWebAPI v1".

The main content area displays the API definition for "EmployeeWebAPI" (version 1.0, OAS 3.0). The URL `https://localhost:7262/swagger/v1/swagger.json` is provided. Below this, the "WeatherForecast" endpoint is listed with a "GET" method. The "Schemas" section is expanded, showing the "WeatherForecast" schema with a right-pointing arrow.



CRÉATION DES CLASSES DU MODÈLE

- Dans le but de pouvoir réutiliser les classes du modèle dans d'autres projets, créer dans votre solution, un nouveau projet de type **bibliothèque de classe** nommé « **MesClasses** » et ajouter les classes suivantes :

```
public class Employee
{
    public int EmployeeId { get; set; }
    [Required]
    [StringLength(100, MinimumLength = 2)]
    public string FirstName { get; set; }
    [Required]
    public string LastName { get; set; }
    [Required]
    [EmailAddress]
    public string Email { get; set; }
    public DateTime DateOfBirth { get; set; }
    public Gender Gender { get; set; }
    public int DepartmentId { get; set; }
    public string PhotoPath { get; set; }
    public Department? Department { get; set; }
}

public enum Gender
{
    Male,
    Female
}
```

```
public class Department
{
    public int DepartmentId { get; set; }
    public string DepartmentName { get; set; }
}
```



INSTALLER LES PACKAGES ENTITY FRAMEWORK CORE

NuGet : EmployeeWebAPI

Parcourir

Installé

Mises à jour 1

microsoft.EntityFrameworkCore

Inclure la versio...

EF

Microsoft.EntityFrameworkCore par aspnet, dotnetframework, EntityFramework, Microsoft, 1,47B téléchargements 9.0.4
Entity Framework Core is a modern object-database mapper for .NET. It supports LINQ queries, change tracking, updates, and schema migrations. EF Core works with SQL Server, Azure SQL Database, SQLite, Azure Cosmos DB, MySQL, PostgreSQL, and other databases through a provider plugi...

EF

Microsoft.EntityFrameworkCore.Abstractions par aspnet, dotnetframework, EntityFramework, Microsoft, 1,44B téléchargements 9.0.4
Provides abstractions and attributes that are used to configure Entity Framework Core

EF

Microsoft.EntityFrameworkCore.Relational par aspnet, dotnetframework, EntityFramework, Microsoft, 1,4B téléchargements 9.0.4
Shared Entity Framework Core components for relational database providers.

EF

Microsoft.EntityFrameworkCore.Analyzers par aspnet, dotnetframework, EntityFramework, Microsoft, 1,38B téléchargements 9.0.4
CSharp Analyzers for Entity Framework Core.

EF

Microsoft.EntityFrameworkCore.Design par aspnet, dotnetframework, EntityFramework, Microsoft, 623M téléchargements 9.0.4
Shared design-time components for Entity Framework Core tools.

EF

Microsoft.EntityFrameworkCore.SqlServer par aspnet, dotnetframework, EntityFramework, Microsoft, 643M téléchargements 9.0.4
Microsoft SQL Server database provider for Entity Framework Core.

EF

Microsoft.EntityFrameworkCore.Tools par aspnet, dotnetframework, EntityFramework, Microsoft, 407M téléchargements 9.0.4
Entity Framework Core Tools for the NuGet Package Manager Console in Visual Studio.

Gestionnaire de package NuGet : EmployeeWebAPI

Source de package : nuget.org

EF

Microsoft.EntityFrameworkCore nuget.org

Version : 8.0.15 Installer

Le mappage de la source du paquet est désactivé. Configurer

Options

Afficher la fenêtre d'aperçu

LISEZMOI Détails du package

Entity Framework Core (EF Core) is a modern object-database mapper that lets you build a clean, portable, and high-level data access layer with .NET (C#) across a variety of databases, including SQL Server (on-premises and Azure), SQLite, MySQL, PostgreSQL, Oracle, and Azure Cosmos DB. It supports LINQ queries, change tracking, updates, and schema migrations.

Getting started

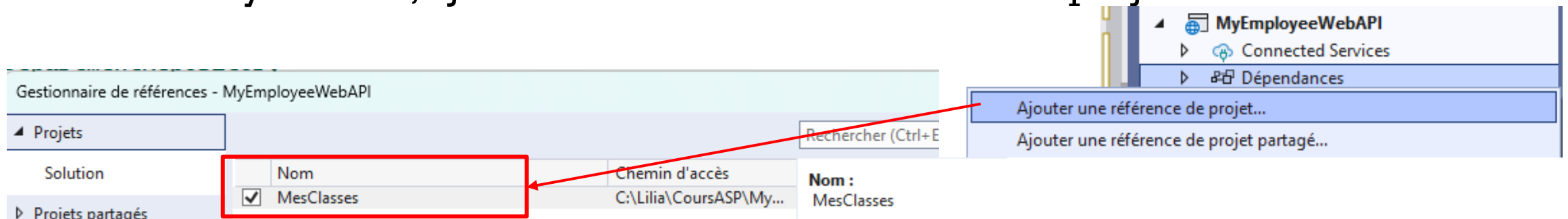
Prerequisites

CRÉER LA CLASSE DE CONTEXTE

- Créer un dossier « Models » dans lequel vous allez ajouter la classe suivante :

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) :base(options)
    {
    }
    public DbSet<Employee> Employees { get; set; }
    public DbSet<Department> Departments { get; set; }
}
```

- Comme étant les classes Employee et Department incluses dans la bibliothèque de classes « MyModel », ajouter la référence convenable au projet



- Ajouter la directive : `using MesClasses;`



CONFIGURATION DE LA BD

- Dans le fichier de configuration appsettings.json, ajouter la chaîne de connexion de la base SQL SERVER

```
"ConnectionStrings": {  
  "DBConnection":  
    "server=(localdb)\\MSSQLLocalDB;database=APIEmployeeDB;Trusted_Connection=true"  
}
```

- Inscrire le service de contexte dans le fichier « Program.cs » :

```
// Add services to the container.  
  
builder.Services.AddControllers();  
// Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle  
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();  
builder.Services.AddDbContext<AppDbContext>(options =>  
    options.UseSqlServer(builder.Configuration.GetConnectionString("DBConnection")));  
var app = builder.Build();
```



EF MIGRATION

- Dans Package Manager Console, Exécuter les commandes de Migration pour créer la base de données :

Add-Migration InitialCreate

Update-Database



REPOSITORY PATTERN INTERFACE

- Dans le dossier Models, Ajouter un dossier Repositories.
- Créer l'interface IEmployeeRepository qui définit les opérations de base sur les employés :

```
public interface IEmployeeRepository
{
    2 références
    Task<IEnumerable<Employee>> GetEmployees();
    4 références
    Task<Employee> GetEmployee(int employeeId);
    2 références
    Task<Employee> AddEmployee(Employee employee);
    2 références
    Task<Employee> UpdateEmployee(Employee employee);
    2 références
    Task<Employee> DeleteEmployee(int employeeId);
    2 références
    Task<Employee> GetEmployeeByEmail(string email);
}
```



REPOSITORY PATTERN – IMPLÉMENTATION SQL SERVER

```
using Microsoft.EntityFrameworkCore;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace EmployeeWebAPI.Models.Repositories
{
    1 référence
    public class EmployeeRepository : IEmployeeRepository
    {
        private readonly AppDbContext appDbContext;

        0 références
        public EmployeeRepository(AppDbContext appDbContext)
        {
            this.appDbContext = appDbContext;
        }

        1 référence
        public async Task<IEnumerable<Employee>> GetEmployees()
        {
            return await appDbContext.Employees.ToListAsync();
        }
    }
}
```

```
1 référence
public async Task<IEnumerable<Employee>> GetEmployees()
{
    return await appDbContext.Employees.ToListAsync();
}

1 référence
public async Task<Employee> GetEmployee(int employeeId)
{
    return await appDbContext.Employees
        .FirstOrDefaultAsync(e => e.EmployeeId == employeeId);
}

1 référence
public async Task<Employee> AddEmployee(Employee employee)
{
    var result = await appDbContext.Employees.AddAsync(employee);
    await appDbContext.SaveChangesAsync();
    return result.Entity;
}
```



REPOSITORY PATTERN – IMPLÉMENTATION SQL SERVER

```
public async Task<Employee> UpdateEmployee(Employee employee)
{
    var result = await appDbContext.Employees
        .FirstOrDefaultAsync(e => e.EmployeeId == employee.EmployeeId);

    if (result != null)
    {
        result.FirstName = employee.FirstName;
        result.LastName = employee.LastName;
        result.Email = employee.Email;
        result.DateOfBrith = employee.DateOfBrith;
        result.Gender = employee.Gender;
        result.DepartmentId = employee.DepartmentId;
        result.PhotoPath = employee.PhotoPath;

        await appDbContext.SaveChangesAsync();

        return result;
    }

    return null;
}
```

```
public async Task<Employee> DeleteEmployee(int employeeId)
{
    var result = await appDbContext.Employees
        .FirstOrDefaultAsync(e => e.EmployeeId == employeeId);
    if (result != null)
    {
        appDbContext.Employees.Remove(result);
        await appDbContext.SaveChangesAsync();
        return result;
    }

    return null;
}
```



REPOSITORY PATTERN INTERFACE

- Créer l'interface IDepartmentRepository qui définit les opérations sur les départements :

```
public interface IDepartmentRepository
{
    IEnumerable<Department> GetDepartments();
    Department GetDepartment(int departmentId);
}
```



REPOSITORY PATTERN – IMPLÉMENTATION SQL SERVER

```
public class DepartmentRepository : IDepartmentRepository
{
    private readonly AppDbContext appDbContext;

    0 références
    public DepartmentRepository(AppDbContext appDbContext)
    {
        this.appDbContext = appDbContext;
    }

    1 référence
    public Department GetDepartment(int departmentId)
    {
        return appDbContext.Departments
            .FirstOrDefault(d => d.DepartmentId == departmentId);
    }

    1 référence
    public IEnumerable<Department> GetDepartments()
    {
        return appDbContext.Departments;
    }
}
```



INJECTION DES SERVICES – CLASSES REPOSITORY

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DBConnection")));

builder.Services.AddScoped<IDepartmentRepository, DepartmentRepository>();
builder.Services.AddScoped<IEmployeeRepository, EmployeeRepository>();

var app = builder.Build();
```



AVANTAGES DU REPOSITORY PATTERN

- Le code est plus propre et plus facile à réutiliser et à maintenir.
- Nous permet de créer des systèmes faiblement couplés.
- Par exemple, si nous voulons que notre application fonctionne avec oracle au lieu de SQL SERVER, implémentez un OracleRepository qui sait lire et écrire dans la base de données Oracle et enregistrez OracleRepository dans le système d'injection de dépendances.
- Dans un projet de test unitaire, il est facile de remplacer un vrai repository par une fausse implémentation pour les tests.



LES CONTRÔLEURS WEBAPI

- Un contrôleur WebAPI est une classe qui hérite de ControllerBase,
- ControllerBase se trouve dans l' espace de noms Microsoft.AspNetCore.Mvc .
- Créez donc un contrôleur qui dérive de la classe Controller si vous créez une application Web MVC.



AJOUTER UN CONTRÔLEUR API

- Dans le dossier « Controllers », ajouter un nouveau contrôleur WebAPI « EmployeeController »

Ajouter un nouvel élément généré automatiquement

▲ Installé

▲ Éléments communs

API

► Blazor

▲ MVC

Affichage

Contrôleur

Pages Razor

Identité

Mise en page



Contrôleur d'API - Vide



Contrôleur d'API avec actions de lecture/écriture



Contrôleur d'API avec actions, utilisant Entity Framework



API avec points de terminaison de lecture/écriture



API avec points de terminaison de lecture/écriture, à l'aide de Entity Framework

Contrôleur d'API - Vide

par Microsoft
v1.0.0.0

Contrôleur d'API vide.

ID : ApiControllerEmptyScaffolder



EMPLOYEECONTROLLER

```
[Route("api/[controller]")]
[ApiController]
public class EmployeesController : ControllerBase
{
    //Injection de dépendance du service IEmployeeRepository
    private readonly IEmployeeRepository employeeRepository;

    public EmployeesController(IEmployeeRepository employeeRepository)
    {
        this.employeeRepository = employeeRepository;
    }
}
```



LA MÉTHODE D'ACTION- GETEMPLOYEES

```
[HttpGet]
public async Task<ActionResult> GetEmployees()
{
    try
    {
        return Ok(await employeeRepository.GetEmployees());
    }
    catch (Exception)
    {
        return StatusCode(StatusCode.Status500InternalServerError,
            "Error retrieving data from the database");
    }
}
```



HTTP STATUS CODES

- Une fois le projet exécuté, nous pouvons vérifier si notre API fonctionne correctement selon le HTTP STATUS CODES suivants qui fournissent au client de l'API, le statut de la requête.

Level 200

200 OK

201 Created

204 No Content

Level 400

400 Bad Request

401 Unauthorized

404 Not Found

Level 500

500 Internal Server Error

501 Not Implemented

503 Service Unavailable



HTTP STATUS CODES

- ASP.NET Core fournit les méthodes d'assistance suivantes pour renvoyer les codes d'état HTTP:

HTTP Status Code

Helper Method

200 **Ok**

201 **Created**

204 **NoContent**

400 **BadRequest**

401 **Unauthorized**

404 **NotFound**

500 **StatusCode**

501 **StatusCode**

503 **StatusCode**



Postman

File Edit View Help

New Import Runner

My Workspace Invite

Refresh

Search

Notifications

Heart

Profile

Upgrade

Filter

History Collections APIs BETA

Save Responses

You haven't sent any requests

Any request you send in this workspace will appear here.

Show me how

DEL http://localhost:65180/api/Emp...

[CONFLICT] GET http://localhost:65...

GET http://localhost:65180/api/Emp...

POST http://localhost:65180/api/E...

No Environment

Untitled Request

Comments

GET

https://localhost:44369/api/Employees

Send Save

Params Authorization Headers (9) Body Pre-request Script Tests Settings

Cookies Code

Query Params

KEY	VALUE	DESCRIPTION	Bulk Edit
Key	Value	Description	

Body Cookies Headers (5) Test Results

Status: 200 OK Time: 93ms Size: 524 B Save Response

Pretty Raw Preview Visualize BETA JSON

```
1 [
2   {
3     "employeeId": 1,
4     "firstName": "ZRIBI",
5     "lastName": "MALEK",
6     "email": "malek_zribi@yahoo.fr",
7     "dateOfBrith": "1976-05-10T00:00:00",
8     "gender": 1,
9     "departmentId": 1,
10    "photoPath": null
11  },
12  {
13    "employeeId": 2,
14    "firstName": "TRIKI",
15    "lastName": "MOHAMED ALI",
16    "email": "triki_ali@gmail.com",
17    "dateOfBrith": "1988-05-22T00:00:00",
18    "gender": 0,
19    "departmentId": 2,
20    "photoPath": null
12
```

Bootcamp Build Browse

Taper ici pour rechercher

00:16

29/11/2020

GETEMPLOYEE(ID)

URI	HTTP Verb	Outcome
.../api/employees	GET	Gets list of employees
.../api/employees/1	GET	Gets employee with Id = 1

```
[HttpGet("{id:int}")]
public async Task<ActionResult<Employee>> GetEmployee(int id)
{
    try
    {
        var result = await employeeRepository.GetEmployee(id);
        if (result == null) return NotFound();
        return result;
    }
    catch (Exception)
    {
        return StatusCode(StatusCodes.Status500InternalServerError,
            "Error retrieving data from the database");
    }
}
```



Filter

History Collections APIs BETA

Save Responses Clear all

Today

GET https://localhost:44369/api/employees/1

DEL http://localhost:65180/api/Empl... [CONFLICT] GET http://localhost:651... GET http://localhost:65180/api/Empl... + ...

No Environment

Comments

Untitled Request

GET https://localhost:44369/api/Employees/1 Send Save

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies Code

Query Params

KEY	VALUE	DESCRIPTION	Bulk Edit
Key	Value	Description	

Body Cookies Headers (5) Test Results Status: 200 OK Time: 1309ms Size: 349 B Save Response

Pretty Raw Preview Visualize BETA JSON

```
1 {
2   "employeeId": 1,
3   "firstName": "ZRIBI",
4   "lastName": "MALEK",
5   "email": "malek_zribi@yahoo.fr",
6   "dateOfBrith": "1976-05-10T00:00:00",
7   "gender": 1,
8   "departmentId": 1,
9   "photoPath": null
10 }
```

MÉTHODE POST – CRÉER UN EMPLOYÉ

```
[HttpPost]
public async Task<ActionResult<Employee>> CreateEmployee([FromBody] Employee employee)
{
    try
    {
        if (employee == null)
            return BadRequest();
        var createdEmployee = await employeeRepository.AddEmployee(employee);

        return CreatedAtAction(nameof(GetEmployee),
            new { id = createdEmployee.EmployeeId }, createdEmployee);
    }
    catch (Exception)
    {
        return StatusCode(StatusCodes.Status500InternalServerError,
            "Error creating new employee record");
    }
}
```

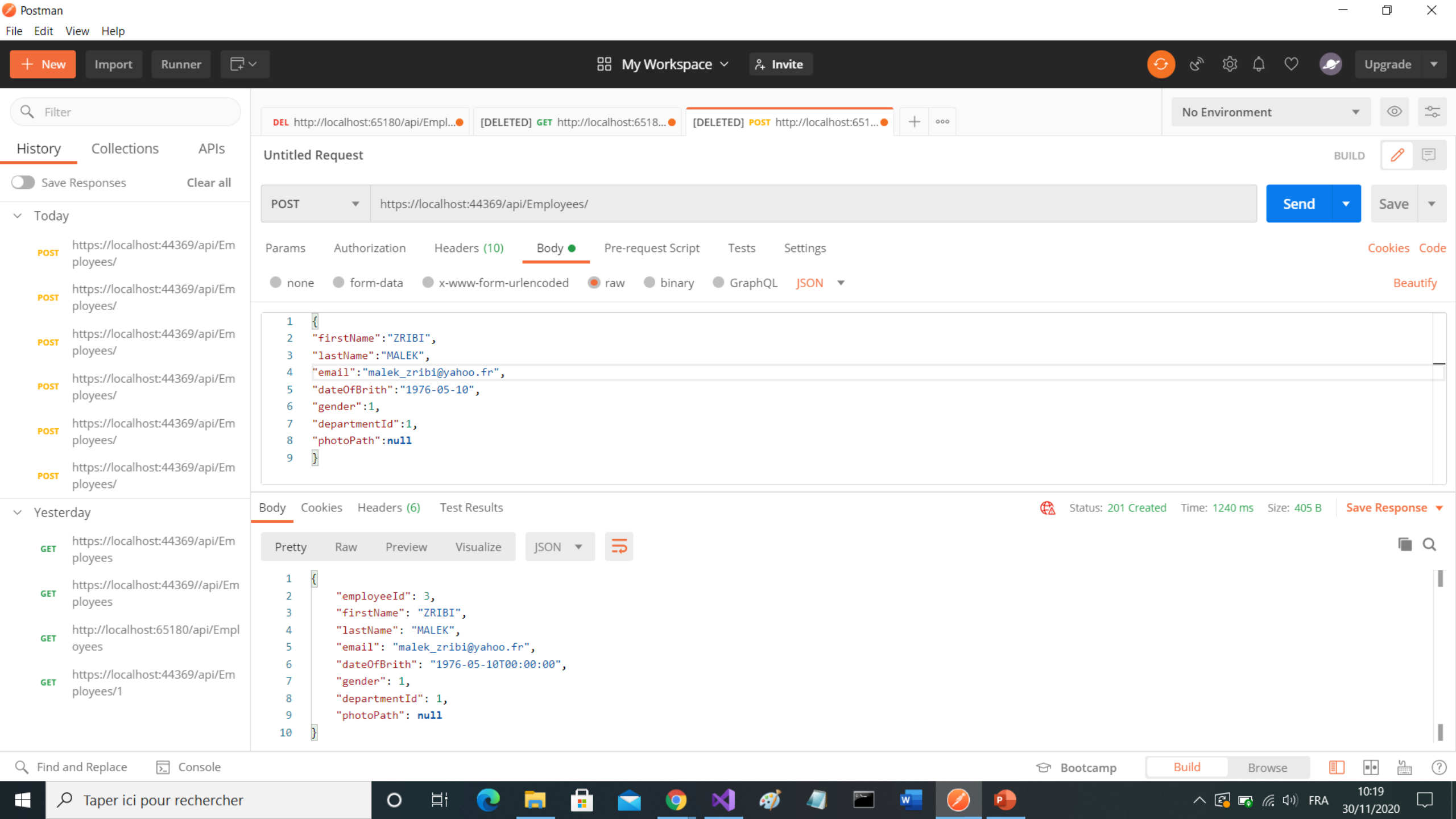


MÉTHODE POST – CRÉER UN EMPLOYÉ

Lorsqu'une nouvelle ressource est créée, les 3 choses suivantes se produisent généralement :

- Renvoyer le code d'état http 201 pour indiquer que la ressource a été créée avec succès.
- Renvoyer la ressource nouvellement créée. Dans notre cas, le nouvel employé.
- Ajoutez un en-tête Location à la réponse. L'en-tête Location spécifie l'URI de l'objet employé nouvellement créé.
- La méthode CreatedAtAction nous aide à réaliser les trois choses ci-dessus. Nous utilisons l'opérateur nameof au lieu d'inclure le nom de la méthode (GetEmployee) dans une chaîne.





VALIDATION DE MODÈLE DANS ASP.NET CORE REST API

- ASP.NET Core fournit plusieurs attributs intégrés pour la validation de modèle
- Required : Spécifie que le champ est obligatoire
- Range : Spécifie la valeur minimale et maximale autorisée
- MinLength : Spécifie la longueur minimale d'une chaîne
- MaxLength : Spécifie la longueur maximale d'une chaîne
- Compare : Compare 2 propriétés d'un modèle. Par exemple, comparer les propriétés Email et ConfirmEmail.
- RegularExpression : Valide si la valeur fournie correspond au modèle spécifié par l'expression régulière
- Ces attributs de validation se trouvent dans l'espace de noms `System.ComponentModel.DataAnnotations`



VALIDATION DE MODÈLE DANS ASP.NET CORE REST API

- Pour implémenter la validation de modèle dans une API REST ASP.NET Core, décorez les propriétés respectives avec les attributs de validation.
- Dans l'exemple suivant, FirstName est une propriété obligatoire. Doit contenir un minimum de 2 caractères et ne doit pas dépasser 100 caractères.

```
public class Employee
{
    5 références
    public int EmployeeId { get; set; }
    [Required]
    [StringLength(100, MinimumLength = 2)]
    2 références
    public string FirstName { get; set; }
    [Required]
    2 références
    public string LastName { get; set; }
    [Required]
    2 références
    public string Email { get; set; }
    2 références
    public DateTime DateOfBrith { get; set; }
    2 références
    public Gender Gender { get; set; }
    2 références
    public int DepartmentId { get; set; }
    2 références
    public string PhotoPath { get; set; }
}
```



- Si on saisit des données non conformes pour les colonnes firstname et lastname on aura les messages d'erreurs suivants :

The screenshot shows the Postman application interface. On the left, the 'History' tab is active, showing a list of requests. The main panel displays an 'Untitled Request' for a POST method to the URL 'https://localhost:44369/api/Employees/'. The request body is a JSON object:

```
1 {
2   "firstName": "B",
3   "email": "malek_zribi@yahoo.fr",
4   "dateOfBrith": "1976-05-10",
5   "gender": 1,
6   "departmentId": 1,
7   "photoPath": null
8 }
```

The response tab is selected, showing a 'Status: 400 Bad Request' with a time of 31 ms and a size of 522 B. The response body is a JSON object with error details:

```
1 {
2   "status": 400,
3   "traceId": "|871a158d-454303cd000a058f.",
4   "errors": {
5     "LastName": [
6       "The LastName field is required."
7     ],
8     "FirstName": [
9       "The field FirstName must be a string with a minimum length of 2 and a maximum length of 100."
10    ]
11  }
12 }
```



ERREURS DE VALIDATION DE MODÈLE

- Pour ajouter une erreur de validation de modèle personnalisé, utilisez la méthode **AddModelError ()** de l'objet **ModelState**.
- Dans une API REST ASP.NET Core, il n'est pas nécessaire de vérifier explicitement si l'état du modèle est valide.
- La classe de contrôleur est décorée avec l'attribut [ApiController], elle prend soin de vérifier si l'état du modèle est valide et renvoie automatiquement l'état 400 (Bad Request) avec les erreurs de validation.
- Apportons à la méthode CreateEmployee les modifications nécessaires pour valider les données envoyés et vérifier si l'email de l'employé existe déjà.

2 références

```
public async Task<Employee> GetEmployeeByEmail(string email)
{
    return await appDbContext.Employees
        .FirstOrDefaultAsync(e => e.Email == email);
}
```




```
public async Task<ActionResult<Employee>> CreateEmployee(Employee employee)
{
    try
    {
        if (employee == null)
        {
            return BadRequest();
        }
        else
        // Add custom model validation error
        {
            var emp = await employeeRepository.GetEmployeeByEmail(employee.Email);
            if (emp != null)
            {
                ModelState.AddModelError("email", "Employee email already in use");
                return BadRequest(ModelState);
            }
            else
            {
                var createdEmployee = await employeeRepository.AddEmployee(employee);

                return CreatedAtAction(nameof(GetEmployee), new { id = createdEmployee.EmployeeId },
                    createdEmployee);
            }
        }
    }
    catch (Exception)
    {
        return StatusCode(StatusCodes.Status500InternalServerError, "Error retrieving data from the database");
    }
}
```



- Si on envoie à l'API un nouvel employé (POST) avec un email existant, on aura comme réponse un Bad Request (400) avec le message d'erreur de validation.

The screenshot shows a REST client interface with a top bar containing several request history items. The main area is titled 'Untitled Request' and shows a POST request to 'https://localhost:44369/api/Employees/'. The request body is a JSON object with employee details. Below the request editor, the response is displayed with a status of '400 Bad Request'. The response body contains an error message: 'Employee email already in use'.

DEL http://localhost:65180/api/Empl... [DELETED] GET http://localhost:6518... [DELETED] POST http://localhost:651...

Untitled Request BUILD

POST https://localhost:44369/api/Employees/ Send Save

Params Authorization Headers (10) Body Pre-request Script Tests Settings Cookies (

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "firstName": "BEN ALI",
3   "lastName": "SOFIEN",
4   "email": "malek_zribi@yahoo.fr",
5   "dateOfBrith": "1976-05-10",
6   "gender": 1,
7   "departmentId": 1,
8   "photoPath": null
9 }
```

Body Cookies Headers (5) Test Results Status: 400 Bad Request Time: 235 ms Size: 234 B Save Respons

Pretty Raw Preview Visualize JSON

```
1 {
2   "email": [
3     "Employee email already in use"
4   ]
5 }
```

METTRE À JOUR LES DONNÉES (LA REQUÊTE HTTP PUT)

- Pour mettre à jour les données d'un employé, ajouter un http put request, et donner dans l'url l'ID de l'employé à modifier.

URI	HTTP Verb	Outcome
.../api/employees	GET	Gets list of employees
.../api/employees/1	GET	Gets employee with Id = 1
.../api/employees	POST	Creates a new employee
.../api/employees/1	PUT	Updates employee with Id = 1

- Le paramètre de la méthode UpdateEmployee doit être décoré avec l'attribut [FromBody] mais ceci n'est pas obligatoire si vous avez ajouté l'attribut [ApiController] au niveau du contrôleur.



METTRE À JOUR LES DONNÉES (LA REQUÊTE HTTP PUT)

```
[HttpPut("{id:int}")]
```

0 références

```
public async Task<ActionResult<Employee>> UpdateEmployee(int id, Employee employee)
{
    try
    {
        if (id != employee.EmployeeId)
            return BadRequest("Employee ID mismatch");

        var employeeToUpdate = await employeeRepository.GetEmployee(id);

        if (employeeToUpdate == null)
            return NotFound($"Employee with Id = {id} not found");

        return await employeeRepository.UpdateEmployee(employee);
    }
    catch (Exception)
    {
        return StatusCode(StatusCodes.Status500InternalServerError,
            "Error updating data");
    }
}
```



METTRE À JOUR LES DONNÉES (LA REQUÊTE HTTP PUT)

Untitled Request

PUT

https://localhost:44369/api/Employees/3

Send

Save

Params

Authorization

Headers (10)

Body

Pre-request Script

Tests

Settings

● none

● form-data

● x-www-form-urlencoded

● raw

● binary

● GraphQL

JSON

1

{

2

"employeeId": 3,

3

"firstName": "Akroute",

4

"lastName": "Salah",

5

"email": "ak_sal@yahoo.fr",

6

"dateOfBrith": "1976-05-10T00:00:00",

7

"gender": 1,

8

"departmentId": 1,

9

"photoPath": null

10

}

Body

Cookies

Headers (5)

Test Results

Status: 200 OK

Time: 1449 ms

Size: 346 B

Save Response

Pretty

Raw

Preview

Visualize

JSON

1

{

2

"employeeId": 3,

3

"firstName": "Akroute",

4

"lastName": "Salah",

5

"email": "ak_sal@yahoo.fr",

6

"dateOfBrith": "1976-05-10T00:00:00",

7

"gender": 1,

8

"departmentId": 1,

9

"photoPath": null

10

}

REQUÊTE DELETE DANS ASP.NET CORE REST API

- Pour supprimer une ressource, envoyez une requête HTTP DELETE à l'URI /api/ Employees/ID . L' ID de l'employé à supprimer doit être passé dans l'URI.

URI	HTTP Verb	Outcome
.../api/employees	GET	Gets list of employees
.../api/employees	POST	Creates a new employee
.../api/employees/1	GET	Gets employee with Id = 1
.../api/employees/1	PUT	Updates employee with Id = 1
.../api/employees/1	DELETE	Deletes employee with Id = 1



REQUÊTE DELETE DANS ASP.NET CORE REST API

```
[HttpDelete("{id:int}")]
```

0 références

```
public async Task<ActionResult<Employee>> DeleteEmployee(int id)
{
    try
    {
        var employeeToDelete = await employeeRepository.GetEmployee(id);

        if (employeeToDelete == null)
        {
            return NotFound($"Employee with Id = {id} not found");
        }

        return await employeeRepository.DeleteEmployee(id);
    }
    catch (Exception)
    {
        return StatusCode(StatusCodes.Status500InternalServerError,
            "Error deleting data");
    }
}
```



Untitled Request

BUILD  

DELETE

https://localhost:44369/api/Employees/3

Send

Save

- Params
- Authorization
- Headers (8)
- Body
- Pre-request Script
- Tests
- Settings
- none
- form-data
- x-www-form-urlencoded
- raw
- binary
- GraphQL
- Cookies
- Cc

This request does not have a body

Pretty

Raw

Preview

Visualize

JSON



```
1 {
2   "employeeId": 3,
3   "firstName": "Akroute",
4   "lastName": "Salah",
5   "email": "ak_sal@yahoo.fr",
6   "dateOfBrith": "1976-05-10T00:00:00",
7   "gender": 1,
8   "departmentId": 1,
9   "photoPath": null
10 }
```



REQUÊTE HTTP SEARCH

- Pour implémenter le service de recherche, on commence par ajouter la méthode Search dans EmployeeRepository.

```
public interface IEmployeeRepository
{
    2 références
    Task<IEnumerable<Employee>> GetEmployees();
    4 références
    Task<Employee> GetEmployee(int employeeId);
    2 références
    Task<Employee> AddEmployee(Employee employee);
    2 références
    Task<Employee> UpdateEmployee(Employee employee);
    2 références
    Task<Employee> DeleteEmployee(int employeeId);
    2 références
    Task<Employee> GetEmployeeByEmail(string email);
    1 référence
    Task<IEnumerable<Employee>> Search(string name, Gender? gender);
}
```



REQUÊTE HTTP SEARCH

- Code de la méthode Search dans EmployeeRepository

```
public async Task<IEnumerable<Employee>> Search(string name, Gender? gender)
{
    IQueryable<Employee> query = appDbContext.Employees;

    if (!string.IsNullOrEmpty(name))
    {
        query = query.Where(e => e.FirstName.Contains(name)
                               || e.LastName.Contains(name));
    }

    if (gender != null)
    {
        query = query.Where(e => e.Gender == gender);
    }

    return await query.ToListAsync();
}
```



REQUÊTE HTTP SEARCH

- Code de la méthode d'action Search dans EmployeesController

```
[HttpGet("{search}")]
```

0 références

```
public async Task<ActionResult<IEnumerable<Employee>>> Search(string name, Gender? gender)
{
    try
    {
        var result = await employeeRepository.Search(name, gender);

        if (result.Any())
        {
            return Ok(result);
        }

        return NotFound();
    }
    catch (Exception)
    {
        return StatusCode(StatusCodes.Status500InternalServerError,
            "Error retrieving data from the database");
    }
}
```



REQUÊTE HTTP SEARCH

[DELETED] PUT http://... ● [DELETED] POST http://... ● POST https://localho... ● DEL https://localhost... ● POST https://localho... ● GET https://localhost... ● + ...

No Environment ▼

👁️ ⚙️

Untitled Request

BUILD



GET ▼

https://localhost:44369/api/Employees/Search?name=zribi&gender=0

Send

Save

Params ● Authorization Headers (7) Body Pre-request Script Tests Settings

Cookies Code

Query Params

	KEY	VALUE	DESCRIPTION	...	Bulk Edit
☒	name	zribi			
☒	gender	0			
	Key	Value	Description		

Body Cookies Headers (5) Test Results



Status: 200 OK

Time: 29 ms

Size: 351 B

Save Response ▼

Pretty Raw Preview Visualize

JSON ▼



```
1  [
2    {
3      "employeeId": 1,
4      "firstName": "ZRIBI",
5      "lastName": "MALEK",
6      "email": "malek_zribi@yahoo.fr",
7      "dateOfBrith": "1976-05-10T00:00:00",
8      "gender": 0,
9      "departmentId": 1,
10     "photoPath": null
11   }
12 ]
```

REQUÊTE HTTP SEARCH

[DELETED] PUT http://... ●

[DELETED] POST http://... ●

POST https://localho... ●

DEL https://localhost... ●

POST https://localho... ●

GET https://localhost... ●

+

...

No Environment ▼

👁

🔗

Untitled Request

BUILD

✎

💬

GET ▼

https://localhost:44369/api/Employees/Search?name=turki

Send ▼

Save ▼

Params ●

Authorization

Headers (7)

Body

Pre-request Script

Tests

Settings

Cookies Co

Query Params

	KEY	VALUE	DESCRIPTION	...	Bulk Ed
☒	name	turki			
	Key	Value	Description		

Body

Cookies

Headers (5)

Test Results

🌐

Status: 404 Not Found

Time: 72 ms

Size: 330 B

Save Response

Pretty

Raw

Preview

Visualize

JSON ▼

☰

📄

🔍

```
1 {
2   "type": "https://tools.ietf.org/html/rfc7231#section-6.5.4",
3   "title": "Not Found",
4   "status": 404,
5   "traceId": "|ba9c9e70-44871144cbef37ad."
}
```